

LAPORAN PRAKTIKUM 4

QUEUE

STRUKTUR DATA



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Queue” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Dalam laporan ini, penulis akan membahas mengenai konsep dan implementasi struktur data *queue*, yaitu struktur data yang bekerja berdasarkan prinsip *First In First Out* (FIFO), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Implementasi queue pada laporan ini dilakukan menggunakan bahasa pemrograman Java.

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dalam penyusunan narasi laporan maupun optimasi kode program di dalamnya. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 28 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	
2.1 Landasan Teori.....	3
2.1.1 Pengertian dan Prinsip Kerja Queue	
2.1.2 Operasi Fundamental pada Queue	
2.1.3 Implementasi Queue pada Java	
2.2 Alat dan Bahan.....	4
2.3 Langkah Praktikum dan Analisis Kode Program.....	4
2.3.1 Kelas QueueArray	
2.3.2 Kelas Driver QueueArray	
2.3.3 Kelas QueueLinkedList	
2.3.4 Kelas ReverseData	
2.3.5 Kelas IterasiQueue	
BAB III PENUTUP	
3.1 Kesimpulan.....	13
3.2 Saran.....	13
DAFTAR PUSTAKA.....	14
TUGAS 4 PRAKTIKUM STRUKTUR DATA.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia komputasi dan ilmu komputer, struktur data merupakan fondasi utama dalam mengelola dan menyimpan data secara efisien. Salah satu struktur data dasar yang sangat sering diimplementasikan dalam berbagai sistem adalah Queue atau antrean. Secara konseptual, Queue merepresentasikan kejadian di dunia nyata di mana entitas yang datang lebih awal akan dilayani terlebih dahulu, seperti antrean pada loket kasir atau antrean pencetakan dokumen pada printer.

Dalam dunia informatika, konsep ini diterapkan pada berbagai proses vital, seperti penjadwalan proses pada CPU, antrean pencetakan dokumen pada printer, hingga pengelolaan transfer paket data dalam jaringan komputer.

Pada bahasa pemrograman Java, Queue dapat diimplementasikan menggunakan berbagai cara, seperti dengan memanfaatkan Library Collection Framework bawaan Java seperti LinkedList untuk pembuatan antrean yang dinamis, maupun membangun logika antrean secara fundamental menggunakan tipe data Array. Mempelajari kedua pendekatan ini secara komprehensif merupakan langkah esensial bagi mahasiswa Informatika. Pemahaman yang mendalam mengenai logika operasi dasar Queue akan menjadi fondasi yang kuat dalam merancang sistem perangkat lunak yang responsif, optimal, dan bebas dari kebocoran memori.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

- 1) Membantu mahasiswa memahami definisi, karakteristik, dan konsep dasar struktur data Queue yang berlandaskan pada prinsip FIFO (First In First Out).
- 2) Melatih kemampuan mahasiswa dalam merancang dan mengimplementasikan algoritma Queue secara manual menggunakan representasi Array statis.

- 3) Melatih kemampuan mahasiswa dalam memanfaatkan Collection Framework Java, khususnya antarmuka Queue dan kelas LinkedList, untuk membangun antrean yang bersifat dinamis.
- 4) Meningkatkan keterampilan analisis mahasiswa dalam membedah logika kode program, serta melakukan modifikasi pada operasi dasar antrean seperti penyisipan (enqueue), penghapusan (dequeue), dan iterasi.

1.3 Manfaat Praktikum

Melalui praktikum ini, mahasiswa diharapkan dapat memperoleh manfaat akademis dan teknis yang signifikan. Secara akademis, mahasiswa mampu mengkorelasikan teori struktur data yang dipelajari di kelas dengan penerapan nyata di dalam kode program. Secara teknis, mahasiswa terlatih untuk berpikir algoritmik, terbiasa melakukan debugging mandiri, serta mampu memilih jenis struktur data (Array atau LinkedList) yang paling efisien berdasarkan studi kasus penyelesaian masalah yang diberikan.

BAB II

PEMBAHASAN

2.1 Landasan Teori

2.1.1. Pengertian dan Prinsip Kerja Queue

Queue (Antrean) adalah suatu bentuk struktur data linier yang berfungsi untuk menyimpan koleksi elemen dengan aturan penambahan dan penghapusan data yang ketat. Queue beroperasi dengan prinsip FIFO (First In First Out), yang secara harfiah berarti "Pertama Masuk, Pertama Keluar". Berbeda dengan Stack (tumpukan) yang ujung masuk dan keluarnya sama, pada Queue, proses modifikasi data dilakukan pada dua ujung yang berbeda. Elemen baru selalu ditambahkan di ujung belakang antrean yang disebut Rear/Tail, sedangkan elemen yang akan diproses atau dihapus selalu diambil dari ujung depan yang disebut Front/Head.

2.1.2. Operasi Fundamental pada Queue

Dalam pengimplementasiannya, struktur data Queue wajib memiliki beberapa fungsi atau metode operasional utama, yaitu:

- Enqueue (Add / Offer): Operasi penambahan satu elemen data ke dalam posisi paling belakang (rear) dari antrean.
- Dequeue (Remove / Poll): Operasi pengambilan dan penghapusan satu elemen data dari posisi paling depan (front) antrean.
- Peek (Front): Operasi inspeksi untuk melihat nilai elemen terdepan pada antrean tanpa menghapus elemen tersebut dari memori.
- IsEmpty: Fungsi validasi Boolean yang digunakan untuk memverifikasi apakah antrean dalam keadaan kosong, yang berguna untuk mencegah eror Underflow.
- IsFull: Fungsi validasi Boolean untuk memverifikasi apakah batas kapasitas memori antrean sudah penuh, untuk mencegah eror Overflow (umumnya digunakan pada antrean berbasis Array).

2.1.3. Implementasi Queue pada Java

Dalam bahasa Java, pendekatan implementasi Queue terbagi menjadi dua paradigma utama:

- 1) Menggunakan Interface `java.util.Queue`: Karena Queue adalah sebuah interface (antarmuka), ia tidak dapat langsung diinstansiasi menjadi objek tunggal. Programmer harus menggunakan kelas turunannya (implementasinya) seperti `LinkedList` atau `PriorityQueue`. Pendekatan ini sangat dinamis; ukuran antrean dapat membesar dan mengecil secara otomatis seiring dengan bertambah atau berkurangnya data.
- 2) Menggunakan Array Kustom: Programmer mendefinisikan batas ukuran (capacity) secara tetap (statis) di awal program. Meskipun ukurannya tidak fleksibel, pendekatan ini sangat baik untuk melatih pemahaman konsep pengelolaan memori, indeks ruang, dan teknik Circular Queue (antrean melingkar).

2.2 Alat dan Bahan

- 1) Perangkat Keras:

Komputer atau laptop yang memadai untuk kompilasi kode Java.

- 2) Perangkat Lunak:

- OS pendukung seperti windows 10/11, macOS, atau distribusi Linux.
- Java Development Kit (JDK) versi 8 atau yang lebih baru sebagai compiler utama.
- Integrated Development Environment (IDE) penunjang seperti Eclipse, IntelliJ IDEA, atau Apache NetBeans untuk penulisan dan pengujian kode.

2.3 Langkah Praktikum dan Analisis Kode Program

2.3.1 Kelas `QueueArray`

Metode ini sangat baik untuk melatih pemahaman pengelolaan memori dan teknik Circular Queue (antrean melingkar).

```
package pekan4_2511532023;  
  
public class QueueArray_2511532023 {
```

```

int front_2023, rear_2023, size_2023;
int capacity_2023;
int array_2023[];

public QueueArray_2511532023(int capacity_2023) {
    this.capacity_2023 = capacity_2023;
    front_2023 = this.size_2023 = 0;
    rear_2023 = capacity_2023 - 1;
    array_2023 = new int[this.capacity_2023];
}

boolean isFull(QueueArray_2511532023 queue) {
    return (queue.size_2023 == queue.capacity_2023);
}

boolean isEmpty(QueueArray_2511532023 queue) {
    return (queue.size_2023 == 0);
}

void enqueue_2023(int item) {
    if (isFull(this))
        return;
    this.rear_2023 = (this.rear_2023 + 1) % this.capacity_2023;
    this.array_2023[this.rear_2023] = item;
    this.size_2023 = this.size_2023 + 1;
    System.out.println(item + " enqueued to queue");
}

int dequeue_2023() {
    if (isEmpty(this))
        return Integer.MIN_VALUE;
    int item_2023 = this.array_2023[this.front_2023];
    this.front_2023 = (this.front_2023 + 1) %
this.capacity_2023;
    this.size_2023 = this.size_2023 - 1;
    return item_2023;
}

int front_2023() {
    if (isEmpty(this))
        return Integer.MIN_VALUE;
    return this.array_2023[this.front_2023];
}

int rear_2023() {
    if (isEmpty(this))
        return Integer.MIN_VALUE;
    return this.array_2023[this.rear_2023];
}

// Mencetak elemen antrian
void display_2023() {
    int i;
    if (front_2023 == rear_2023) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    // Kunjungi dari belakang dan cetak
    for (i = front_2023; i < rear_2023; i++) {
        System.out.printf(" %d <-- ", array_2023[i]);
    }
}

```



```

    }
    return;
}
}

```

- `int front_2023, rear_2023, size_2023;`: Mendeklarasikan penunjuk indeks depan (*front*), indeks belakang (*rear*), dan jumlah data aktual (*size*).
- `int capacity_2023;`: Variabel untuk menyimpan batas maksimal tampungan array.
- `public QueueArray_2511532023(int capacity_2023) {`: Ini adalah *Constructor*. Berfungsi menyiapkan kapasitas awal ketika objek diciptakan.
- `front_2023 = this.size_2023 = 0;`: Di awal terbentuk, penunjuk depan dan total isi diatur ke angka 0.
- `rear_2023 = capacity_2023 - 1;`: Penunjuk belakang disetel di indeks terakhir sebagai persiapan agar saat ditambah 1, ia kembali ke indeks 0 berkat operasi modulus.
- `array_2023 = new int[this.capacity_2023];`: Mengalokasikan ruang memori fisik array sesuai kapasitas.
- `boolean isFull(...)` dan `isEmpty(...)`: Fungsi validasi untuk mengecek apakah ukuran saat ini (*size*) sama dengan batas kapasitas (penuh) atau sama dengan 0 (kosong).
- `void enqueue_2023(int item) {`: Fungsi utama untuk memasukkan data.
- `this.rear_2023 = (this.rear_2023 + 1) % this.capacity_2023;`: Ini adalah rumus antrean melingkar (*Circular Queue*). Posisi belakang ditambah 1, lalu dicari sisa baginya terhadap kapasitas. Hal ini mencegah indeks keluar batas (*Out of Bounds*).
- `int dequeue_2023() {`: Fungsi utama untuk mengeluarkan data.
- `this.front_2023 = (this.front_2023 + 1) % this.capacity_2023;`: Sama seperti saat memasukkan data, posisi depan digeser maju dengan rumus modulus agar pergerakannya melingkar.

- for (i = front_2023; i < rear_2023; i++): Pada fungsi display_2023, perulangan ini bertugas mencetak isi array mulai dari titik indeks paling depan hingga titik belakang secara horizontal.

2.3.2 Kelas Driver QueueArray

```
package pekan4_2511532023;

public class QueueArrayDriver_2511532023 {
    public static void main(String[] args) {
        QueueArray_2511532023 queue_2023 = new
        QueueArray_2511532023(1000);
        queue_2023.enqueue_2023(10);
        queue_2023.enqueue_2023(20);
        queue_2023.enqueue_2023(30);
        queue_2023.enqueue_2023(40);
        System.out.println("Item di depan " +
        queue_2023.front_2023());
        System.out.println("Item paling belakang " +
        queue_2023.rear_2023());
        System.out.println("Tampilan queue");
        queue_2023.display_2023();
        System.out.println();
        System.out.println(queue_2023.dequeue_2023() + " dihapus
        dari queue");
        System.out.println("Item di depan: " +
        queue_2023.front_2023());
        System.out.println("Item di belakang: " +
        queue_2023.rear_2023());
        System.out.println("Tampilan queue setelah satu data
        dihapus");
        queue_2023.display_2023();
    }
}
```

- QueueArray_2511532023 queue_2023 = new QueueArray_2511532023(1000);: Memanggil kelas logika yang telah dibuat sebelumnya dan memberikan nilai parameter 1000 sebagai kapasitas array maksimum.
- queue_2023.enqueue_2023(10); hingga 40: Menambahkan empat data berturut-turut ke dalam memori array.
- System.out.println(queue_2023.dequeue_2023() + " dihapus dari queue");: Memanggil perintah *dequeue* yang otomatis mengambil angka yang pertama kali masuk (yakni 10), lalu program mencetak log penghapusannya.

Output:

```
Console X Git Staging
<terminated> QueueArrayDriver_2511532023 [Java Application] C
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
Tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan: 20
Item di belakang: 40
Tampilan queue setelah satu data dihapus
20 <-- 30 <--
```

2.3.3 Kelas QueueLinkedList

Kode program ini fokus pada cara penggunaan metode-metode otomatis yang telah disediakan oleh antarmuka Queue dalam Java. Metode ini sangat efisien untuk pembangunan aplikasi secara cepat tanpa perlu memikirkan batas memori.

```
package pekan4_2511532023;

import java.util.Queue;
import java.util.LinkedList;

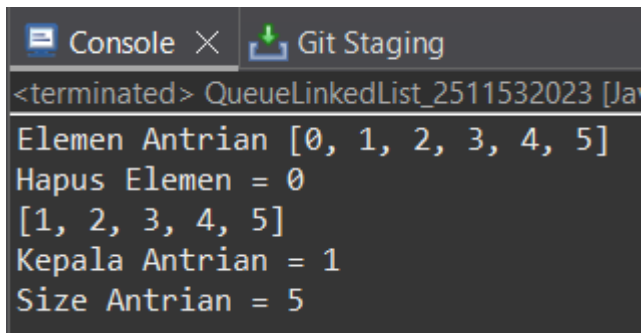
public class QueueLinkedList_2511532023 {
    public static void main(String[] args) {
        Queue<Integer> q_2023 = new LinkedList<>();
        // Tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
        for (int i = 0; i < 6; i++)
            q_2023.add(i);
        // Menampilkan isi antrian
        System.out.println("Elemen Antrian " + q_2023);
        // Untuk menghapus kepala antrian
        int hapus_2023 = q_2023.remove();
        System.out.println("Hapus Elemen = " + hapus_2023);
        System.out.println(q_2023);
        // Untuk melihat antrian terdepan
        int depan_2023 = q_2023.peek();
        System.out.println("Kepala Antrian = " + depan_2023);

        int banyak_2023 = q_2023.size();
        System.out.println("Size Antrian = " + banyak_2023);
    }
}
```

```
}
```

- `Queue<Integer> q_2023 = new LinkedList<>();`: Sama seperti sebelumnya, perintah ini membentuk objek antrean dinamis.
- `for (int i = 0; i < 6; i++)`: Menggunakan struktur perulangan `for` agar penulisan kode penambahan data lebih ringkas. Nilai `i` akan bergerak dari 0 hingga 5.
- `q_2023.add(i);`: Memasukkan setiap nilai `i` ke bagian belakang antrean.
- `int hapus_2023 = q_2023.remove();`: Metode `remove()` mengeksekusi operasi *dequeue*. Elemen paling depan (angka 0) dicabut dari antrean dan nilainya disimpan ke dalam variabel `hapus_2023`.
- `int depan_2023 = q_2023.peek();`: Metode `peek()` menilik elemen yang saat ini menempati posisi terdepan (setelah 0 dihapus, maka posisi terdepan adalah 1) tanpa menghapusnya dari struktur. Nilai ini ditampung di `depan_2023`.
- `int banyak_2023 = q_2023.size();`: Metode `size()` menghitung jumlah total elemen yang masih tersisa di dalam antrean (dalam kasus ini ada 5 elemen tersisa) dan menyimpannya ke variabel `banyak_2023`.

Output:



```
<terminated> QueueLinkedList_2511532023 [Ja
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

2.3.4 Kelas ReverseData

Kode program ini mendemonstrasikan algoritma kompleksitas rendah di mana kita dapat membalikkan urutan data di dalam antrean dengan memanfaatkan sifat LIFO (Last In First Out) dari Stack.

```

package pekan4_2511532023;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532023 {
    public static void main(String[] args) {
        Queue<Integer> q_2023 = new LinkedList<Integer>();
        q_2023.add(1);
        q_2023.add(2);
        q_2023.add(3); // [1, 2, 3]
        System.out.println("Sebelum reverse = " + q_2023);
        Stack<Integer> s_2023 = new Stack<Integer>();
        while (!q_2023.isEmpty()) { // Q -> S
            s_2023.push(q_2023.remove());
        }
        while (!s_2023.isEmpty()) { // S -> Q
            q_2023.add(s_2023.pop());
        }
        System.out.println("Sesudah reverse = " + q_2023); // [3,
2, 1]
    }
}

```

- `q_2023.add(1);` hingga `q_2023.add(3);`: Memasukkan angka 1, 2, dan 3 secara berurutan ke dalam antrean (proses *enqueue*).
- `System.out.println(...);`: Mencetak susunan antrean awal sebelum dimanipulasi ke layar konsol.
- `Stack<Integer> s_2023 = new Stack<Integer>();`: Membuat objek tumpukan baru bernama `s_2023` sebagai wadah penyimpanan sementara.
- `while (!q_2023.isEmpty()) {`: Membuka perulangan `while` yang akan terus berjalan selama antrean `q_2023` tidak kosong.
- `s_2023.push(q_2023.remove());`: Ini adalah inti logika pertama. Data terdepan dari antrean dihapus menggunakan `remove()`, lalu data tersebut langsung ditumpuk ke dalam Stack menggunakan perintah `push()`.
- `while (!s_2023.isEmpty()) {`: Membuka perulangan kedua yang akan terus berjalan selama tumpukan `s_2023` tidak kosong.
- `q_2023.add(s_2023.pop());`: Ini adalah inti logika kedua. Data teratas dari Stack dicabut menggunakan `pop()` (yang mana angka

terakhir yang masuk kini keluar pertama), lalu dimasukkan kembali ke antrean menggunakan add().

- `System.out.println(...);`: Menampilkan hasil akhir antrean yang kini urutannya telah terbalik menjadi [3, 2, 1].

Output:

```
Sebelum reverse = [1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

2.3.5 Kelas IterasiQueue

Objek Iterator merupakan pendekatan tingkat lanjut yang disarankan dalam Java untuk menelusuri isi koleksi secara aman.

```
package pekan4_2511532023;

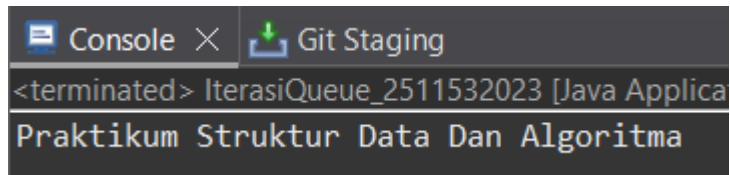
import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;

public class IterasiQueue_2511532023 {
    public static void main(String[] args) {
        Queue<String> q_2023 = new LinkedList<>();
        q_2023.add("Praktikum");
        q_2023.add("Struktur");
        q_2023.add("Data");
        q_2023.add("Dan");
        q_2023.add("Algoritma");
        Iterator<String> iterator_2023 = q_2023.iterator();
        while (iterator_2023.hasNext()) {
            System.out.print(iterator_2023.next() + " ");
        }
    }
}
```

- `Iterator<String> iterator_2023 = q_2023.iterator();`: Menginstansiasi objek Iterator khusus untuk *String*. Objek ini bertindak seperti kursor pembaca yang menautkan dirinya pada antrean `q_2023`.
- `while (iterator_2023.hasNext()) {`: Membuka perulangan while. Metode `hasNext()` berfungsi mengecek apakah masih ada elemen selanjutnya di dalam antrean. Jika masih ada, kondisi bernilai `True` dan perulangan dilanjutkan.

- `System.out.print(iterator_2023.next() + " ");` Metode `next()` mengeksekusi kursor untuk mengambil teks pada posisi saat ini, lalu otomatis melangkah ke elemen berikutnya. Teks tersebut kemudian dicetak bersebelahan (karena memakai `print`, bukan `println`).

Output:



```
Console × Git Staging
<terminated> IterasiQueue_2511532023 [Java Applica
Praktikum Struktur Data Dan Algoritma
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan serangkaian kegiatan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa struktur data Queue merupakan komponen fundamental dalam pemrograman yang sangat efektif untuk menangani proses data dengan prinsip antrean First In First Out (FIFO). Melalui implementasi menggunakan Collection Framework Java, terlihat bahwa penggunaan kelas LinkedList memberikan fleksibilitas yang tinggi karena kapasitas memori bersifat dinamis dan pengoperasiannya lebih sederhana berkat metode bawaan seperti `add()`, `remove()`, dan `peek()`.

Di sisi lain, implementasi Queue menggunakan Array secara manual memberikan pemahaman mendalam mengenai manajemen memori statis dan logika indeks melingkar yang efisien. Praktikum ini juga membuktikan bahwa Queue dapat dikolaborasikan dengan struktur data lain, seperti Stack, untuk memanipulasi urutan data secara sistematis, serta dapat ditelusuri dengan aman menggunakan objek Iterator. Secara keseluruhan, penguasaan terhadap Queue sangat krusial bagi mahasiswa Informatika dalam merancang algoritma penjadwalan maupun sistem simulasi di dunia nyata yang membutuhkan keteraturan pemrosesan data.

3.2 Saran

Untuk pengembangan kemampuan dalam praktikum selanjutnya, mahasiswa disarankan untuk lebih memperdalam logika operasional Circular Queue berbasis Array, terutama dalam memahami peran operator modulus untuk mencegah terjadinya kesalahan indeks memori. Selain itu, sangat dianjurkan bagi mahasiswa untuk mulai menerapkan blok penanganan pengecualian (Exception Handling) seperti try-catch pada setiap operasi pengambilan data untuk menjamin keamanan program saat menghadapi kondisi antrean kosong. Eksplorasi lebih lanjut terhadap variasi Queue lainnya, seperti PriorityQueue atau Double-Ended Queue (Deque), juga sangat disarankan guna memperluas wawasan mengenai struktur data yang lebih kompleks.

DAFTAR PUSTAKA

- [1] Kontributor Scribd, "Queue Pada Java," *Scribd*. [Daring]. Tersedia: <https://id.scribd.com/document/414090643/Queue-Pada-Java>. [Diakses: 28-Apr-2026].
- [2] Studocu, "Laporan Praktikum Queue," *Studocu*. [Daring]. Tersedia: <https://www.studocu.id/id/document/universitas-trunojoyo-madura/algoritma-pemrograman/laporan-praktikum-queue/45027851>. [Diakses: 28-Apr-2026].
- [3] Academia, "Laporan Praktikum 5 Algoritma Struktur Data Queue," *Academia.edu*. [Daring]. Tersedia: https://www.academia.edu/32592023/LAPORAN_PRAKTIKUM_5_ALGORITMA_STRUKTUR_DATA_QUEUE. [Diakses: 30-Apr-2026].
- [4] SlideShare, "Implementasi Queue," *SlideShare*. [Daring]. Tersedia: <https://www.slideshare.net/slideshow/implementasi-queue/22867854>. [Diakses: 30-Apr-2026].
- [5] StudyPool, "Laporan Struktur Data Tentang Queue," *StudyPool*. [Daring]. Tersedia: <https://www.studypool.com/documents/7094579/laporan-struktur-data-tentang-queue>. [Diakses: 30-Apr-2026].

TUGAS 4 PRAKTIKUM STRUKTUR DATA

1) Soal

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian)

menggunakan array dalam bahasa Java.

Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan.

Setiap

pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO (First In First Out).

Program harus mampu:

- Menambahkan pelanggan ke antrian (enqueue)
- Menghapus pelanggan dari antrian (dequeue)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

2) Jawab

```
1. import java.util.Scanner;
2.
3. public class AntrianLoket_2511532023 {
4.     // 1. Deklarasi Variabel (Sesuai ketentuan, ditambah
    akhiran _2023)
5.     int max_2023; // Kapasitas maksimal array
6.     int front_2023; // Indeks depan
7.     int rear_2023; // Indeks belakang
8.     String[] queue_2023; // Array penyimpan data pelanggan
    bertipe Teks (String)
9.
10.    // Constructor untuk inisialisasi antrean
11.    public AntrianLoket_2511532023(int ukuran_2023) {
12.        max_2023 = ukuran_2023;
13.        queue_2023 = new String[max_2023];
14.        front_2023 = 0; // Depan selalu di indeks 0
15.        rear_2023 = -1; // Belakang disetel -1 sebagai
        tanda kosong
16.    }
17.
18.    // Method mengecek apakah antrean kosong
19.    boolean isEmpty_2023() {
20.        return rear_2023 == -1;
21.    }
22.
23.    // Method mengecek apakah antrean penuh
```

```

24.         boolean isFull_2023() {
25.             return rear_2023 == max_2023 - 1;
26.         }
27.
28.         // Method menambah data (Enqueue)
29.         void enqueue_2023(String nama_2023) {
30.             if (isFull_2023()) {
31.                 System.out.println("Antrian penuh!");
32.             } else {
33.                 rear_2023++; // Posisi belakang mundur 1
34.                 langkah
35.                 queue_2023[rear_2023] = nama_2023; // Masukkan
36.                 nama pelanggan ke posisi tersebut
37.                 System.out.println("Data berhasil ditambahkan
38.                 ke antrian");
39.             }
40.         }
41.
42.         // Method menghapus data terdepan (Dequeue) dengan
43.         logika Geser Kiri
44.         void dequeue_2023() {
45.             if (isEmpty_2023()) {
46.                 System.out.println("Antrian kosong!");
47.             } else {
48.                 String dihapus_2023 = queue_2023[front_2023];
49.                 // Ambil nama di posisi 0
50.                 System.out.println(dihapus_2023 + " telah
51.                 dilayani");
52.
53.                 // Logika sangat sederhana: Geser semua data
54.                 di belakang maju 1 langkah
55.                 for (int i = 0; i < rear_2023; i++) {
56.                     queue_2023[i] = queue_2023[i + 1];
57.                 }
58.                 rear_2023--; // Kurangi posisi belakang karena
59.                 total orang berkurang
60.             }
61.         }
62.
63.         // Method menampilkan isi antrian
64.         void display_2023() {
65.             if (isEmpty_2023()) {
66.                 System.out.println("Antrian kosong!");
67.             } else {
68.                 System.out.println("Isi antrian:");
69.                 for (int i = 0; i <= rear_2023; i++) {
70.                     System.out.println((i + 1) + ". " +
71.                     queue_2023[i]); // i+1 agar nomor mulai dari 1
72.                 }
73.             }
74.         }
75.     }
76. }

```

```

67.         // Method membalikkan posisi (Reverse) antrean
68.         void reverse_2023() {
69.             if (isEmpty_2023()) {
70.                 System.out.println("Antrian kosong!");
71.             } else {
72.                 int awal_2023 = 0;
73.                 int akhir_2023 = rear_2023;
74.                 // Logika tukar menukar (Swap) dari ujung ke
ujung menuju tengah
75.                 while (awal_2023 < akhir_2023) {
76.                     String temp_2023 = queue_2023[awal_2023];
77.                     queue_2023[awal_2023] =
queue_2023[akhir_2023];
78.                     queue_2023[akhir_2023] = temp_2023;
79.                     awal_2023++;
80.                     akhir_2023--;
81.                 }
82.                 // Sesuai contoh output tugas, setelah Reverse
langsung tampilkan data
83.                 display_2023();
84.             }
85.         }
86.
87.         // Method utama (Main) untuk menjalankan menu program
88.         public static void main(String[] args) {
89.             Scanner input_2023 = new Scanner(System.in);
90.             // Kita buat kapasitas antrean 10 orang (bisa
diubah sesuai kebutuhan)
91.             AntrianLoket_2511532023 loket_2023 = new
AntrianLoket_2511532023(10);
92.             int pilihan_2023 = 0;
93.
94.             while (pilihan_2023 != 5) {
95.                 System.out.println("\n=== PROGRAM ANTRIAN
LOKET ===");
96.                 System.out.println("1. Tambah Antrian");
97.                 System.out.println("2. Hapus Antrian");
98.                 System.out.println("3. Tampilkan Antrian");
99.                 System.out.println("4. Reverse");
100.                System.out.println("5. Keluar");
101.                System.out.print("Pilih menu: ");
102.                pilihan_2023 = input_2023.nextInt();
103.                input_2023.nextLine(); // Membersihkan buffer
enter bawaan Scanner
104.
105.                if (pilihan_2023 == 1) {
106.                    System.out.print("Masukkan nama pelanggan:
");
107.                    String nama_2023 = input_2023.nextLine();
108.                    loket_2023.enqueue_2023(nama_2023);
109.                } else if (pilihan_2023 == 2) {
110.                    loket_2023.dequeue_2023();

```

```

111.         } else if (pilihan_2023 == 3) {
112.             loket_2023.display_2023();
113.         } else if (pilihan_2023 == 4) {
114.             loket_2023.reverse_2023();
115.         } else if (pilihan_2023 == 5) {
116.             System.out.println("Program selesai");
117.         } else {
118.             System.out.println("Pilihan tidak
valid!");
119.         }
120.     }
121.     input_2023.close();
122. }
123. }

```

- Deklarasi & Persiapan (Constructor): Program menyiapkan sebuah array bernama `queue_2023` yang diibaratkan sebagai kursi ruang tunggu. Posisi kursi paling depan (`front_2023`) dipatok mati selalu di angka 0, sedangkan kursi belakang (`rear_2023`) disetel di -1 yang menandakan ruangan masih kosong sepenuhnya.
- `enqueue_2023` (Tambah Antrean): Saat ada pelanggan baru, program mengecek apakah kursi sudah penuh. Jika belum, batas kursi belakang (`rear_2023`) ditambah 1, lalu nama pelanggan tersebut didudukkan di kursi baru tersebut.
- `dequeue_2023` (Hapus / Layani): Ini adalah inti logika geser kirinya. Pelanggan yang duduk di kursi paling depan (indeks 0) dipanggil untuk dilayani. Karena kursi depan sekarang kosong, sebuah perulangan `for` digunakan untuk menyuruh semua pelanggan di belakangnya untuk bergeser maju satu kursi. Batas kursi belakang (`rear_2023`) kemudian dikurangi 1 karena jumlah orang di ruangan berkurang.
- `display_2023` (Tampilkan Data): Program sekadar menggunakan perulangan untuk menyebutkan nama pelanggan satu per satu, mulai dari kursi nomor 0 sampai batas kursi belakang saat ini.
- `reverse_2023` (Balik Urutan): Program menggunakan teknik swap (tukar-menukar). Pelanggan di kursi paling depan ditukar posisinya dengan pelanggan di kursi paling belakang. Kemudian, pelanggan di

kursi nomor dua ditukar dengan kursi nomor dua dari belakang, begitu seterusnya hingga proses tukar-menukar bertemu di tengah barisan.

- main (Menu Interaktif): Ini adalah otak antarmuka programnya. Menggunakan bantuan Scanner dan perulangan while, layar akan terus memunculkan menu angka 1 sampai 5 dan menunggu kamu mengetik pilihan, lalu menjalankan fungsi-fungsi di atas sesuai angka yang diketik.

Output:

- Tambah Antrain

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian
```

- Tampilkan Antrian

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi
```

- Reverse Data

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Isi antrian:
1. Budi
2. Andi

```

- Hapus Antrian

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

```

- Tampilkan kembali antrian

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi

```

- Keluar dari program

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai

```

Maaf uda laptop saya tadi tiba-tiba rusak, jadi ini pakai laptop lain bikin tugasnya uda,
untuk tugasnya tadi saya bikin pakai web onlinegdp.com uda, jadi tampilannya agak beda,
maaf telat uda... TT